Manipuler des tableaux d'objets

forEach, map et filter au refuge fantastique

Bienvenue au refuge des Brumes ! Pixel le dragon éternue des étincelles et Mochi le slime s'endort dans les chaussures. Votre mission : aider les soigneurs à organiser la vie de ces pensionnaires avec JavaScript.

Six exercices progressifs pour parcourir, transformer et filtrer un tableau d'objets, puis chaîner ces opérations. Travaillez dans la console du navigateur ou dans un fichier refuge.js exécuté avec Node.js.

Le tableau commun aux six missions

```
const creatures = [
  { id: 1, nom: "Pixel", espece: "dragon",
    energie: 80, adoptable: true },
  { id: 2, nom: "Mochi", espece: "slime",
    energie: 25, adoptable: true },
  { id: 3, nom: "Nova", espece: "phenix",
    energie: 95, adoptable: false },
  { id: 4, nom: "Biscotte", espece: "griffon",
    energie: 40, adoptable: true },
  { id: 5, nom: "Nimbus", espece: "dragon",
    energie: 60, adoptable: false },
  { id: 6, nom: "Pistache", espece: "licorne",
    energie: 70, adoptable: true }
];
```

energie est un entier de 0 à 100. adoptable vaut true si l'adoption est autorisée, false sinon. Les espèces sont écrites sans accent dans les données.

Les règles du jeu

Repartez du tableau initial à chaque exercice et conservez son ordre. Ne modifiez ni creatures ni ses objets. Utilisez les méthodes demandées, sans boucle for ou while et sans écrire les résultats en dur.

Affichez vos résultats avec console.log(nomDuTableau) ou console.log(message). La présentation des tableaux et des objets peut varier entre Node.js et le navigateur. Dans le navigateur, dépliez-les si nécessaire : vérifiez surtout leur contenu et leur ordre.

MISSION 1 • forEach • Découverte

Faire l’appel des pensionnaires

Le soigneur commence sa tournée et veut lire une ligne par créature.

1. Parcourez creatures avec forEach.

2. Affichez son nom, son espèce et son énergie au format exact présenté ci-dessous.

CONSOLE • Résultat attendu

```
Pixel (dragon) - énergie : 80/100
Mochi (slime) - énergie : 25/100
Nova (phenix) - énergie : 95/100
Biscotte (griffon) - énergie : 40/100
Nimbus (dragon) - énergie : 60/100
Pistache (licorne) - énergie : 70/100
```

MISSION 2 • map • Extraction

Préparer les étiquettes des casiers

Chaque pensionnaire a besoin d’un casier à son nom.

1. Utilisez map pour créer un tableau noms contenant uniquement les noms des six créatures. Chaque élément doit être une chaîne de caractères.

2. Affichez le tableau avec console.log(noms).

CONSOLE • Résultat attendu

```
[ 'Pixel', 'Mochi', 'Nova', 'Biscotte', 'Nimbus', 'Pistache' ]
```

Point de contrôle : le tableau noms doit contenir autant d’éléments que creatures. Le tableau initial contient toujours les objets complets.

MISSION 3 • filter • Sélection

Repérer les pensionnaires fatigués

Une collation est prévue pour les créatures dont l'énergie est strictement inférieure à 50.

1. Utilisez filter pour créer un tableau fatigues contenant les objets complets des créatures concernées.

2. Affichez le tableau avec console.log(fatigues). Vérifiez qu'il contient les deux objets complets présentés ci-dessous.

CONSOLE • Résultat attendu

```
[
  { id: 2, nom: 'Mochi', espece: 'slime',
    energie: 25, adoptable: true },
  { id: 4, nom: 'Biscotte', espece: 'griffon',
    energie: 40, adoptable: true }
]
```

MISSION 4 • map • Nouveaux objets

Simuler une potion d'énergie

Une potion ajouterait 20 points d'énergie à chaque pensionnaire, sans jamais dépasser 100.

1. À partir de creatures, créez avec map un tableau simulations. Pour chaque créature, retournez un nouvel objet contenant exactement nom et energieApresPotion, dans cet ordre.

2. Calculez la nouvelle énergie en respectant le plafond de 100. Vous pouvez utiliser Math.min(valeur, plafond).

3. Affichez le tableau avec console.log(simulations). Vérifiez les deux propriétés de chaque nouvel objet.

CONSOLE • Résultat attendu

```
[
  { nom: 'Pixel', energieApresPotion: 100 },
  { nom: 'Mochi', energieApresPotion: 45 },
  { nom: 'Nova', energieApresPotion: 100 },
  { nom: 'Biscotte', energieApresPotion: 60 },
  { nom: 'Nimbus', energieApresPotion: 80 },
  { nom: 'Pistache', energieApresPotion: 90 }
]
```

MISSION 5 • filter → map • Chaînage

Organiser la rencontre avec les adoptants

Pour participer à la rencontre, une créature doit être adoptable ET avoir une énergie supérieure ou égale à 60.

1. En une seule expression, chaînez filter puis map à partir de creatures : sélectionnez les créatures qui respectent les deux conditions, puis extrayez leurs noms.

2. Stockez le tableau de chaînes obtenu dans invites, sans tableau intermédiaire, puis affichez-le avec console.log(invites).

CONSOLE • Résultat attendu

```
[ 'Pixel', 'Pistache' ]
```

MISSION 6 • filter → map → forEach • Défi final

Publier les annonces de la rencontre

Le refuge veut maintenant préparer et afficher une annonce pour chaque participant à la rencontre.

1. Repartez de creatures. Dans une seule chaîne, gardez les créatures adoptables dont l'énergie est supérieure ou égale à 60.

2. Transformez chacune des créatures retenues en message avec map. Reproduisez exactement le format de la sortie attendue.

3. Terminez la chaîne avec forEach pour afficher chaque message sur une ligne. N'utilisez pas de tableau intermédiaire.

CONSOLE • Résultat attendu

```
À adopter : Pixel, dragon - énergie 80/100
À adopter : Pistache, licorne - énergie 70/100
```

Avant de rendre votre travail

Vérifiez les six sorties et le respect des méthodes imposées. Sachez expliquer le type des éléments après chaque étape du défi : objets sélectionnés, puis chaînes de caractères. forEach renvoie undefined : il termine ici le chaînage.